Megasub Mobile — Backend Requirements

Push Notifications (iOS & Android) and Continue with Google / Apple

Prepared for the Mega-Sub backend team · From the mobile (React Native / Expo) team · 17 October 2026 · v1.0

What this document is. A list of everything the mobile app needs from the backend to ship (1) push notifications on iOS and Android and (2) "Continue with Google" and "Continue with Apple". It follows the conventions the app already uses with `https://mega-sub.com/api/v1/external`. Items are marked P0 required P1 strongly recommended P2 nice to have.

Contents

1. Shared API conventions
2. Push notifications: architecture
3. Push: device token endpoints
4. Push: what triggers a notification
5. Push: payload format
6. Push: sending, receipts and cleanup
7. Push: preferences and in-app inbox
8. Push: credentials and setup
9. Continue with Google (existing `/google`)
10. Continue with Apple (new `/apple`)
11. Shared rules for social accounts
12. Database changes
13. Open questions
14. Acceptance checklist

1. Shared API conventions (keep these)

The app's networking layer (`lib/api.js`) already assumes the following. New endpoints must follow them too, or the app will mishandle responses.

- **Base URL:** `https://mega-sub.com/api/v1/external`
- **Auth:** `Authorization: Bearer <session token>`. The token comes from `access.token` in the login/social-login response.
- **Envelope:** every response is JSON with `status` (boolean), `message` (readable text the app can show), optionally `code` (number) and `data`. The app treats `status: false` as failure even when HTTP is 200.
- **401 means the session is gone.** Mega-Sub allows one active login per user, so a 401 makes the app log the user out. Only return 401 for a genuinely invalid or replaced token, never for validation errors. Use 422 for those.
- **Never log or echo** push tokens, Google/Apple ID tokens, authorization codes, PINs or passwords in error responses.

2. Push notifications: architecture

The app is built with Expo (SDK 54, EAS project `fc4f4a94-3f72-4021-affe-9420f8230609`) and already includes `expo-notifications`. We recommend **Expo Push Service** as the delivery layer.

Option	How it works	Verdict
A. Expo Push Service (recommended)	The app sends one token per device (`ExponentPushToken[...]`). The backend makes one HTTPS POST to `https://exp.host/--/api/v2/push/send` . Expo forwards it to APNs (iOS) or FCM (Android) using credentials stored in the EAS project.	One API for both platforms, no Firebase or APNs SDK on the server, any language can call it. **Use this.**
B. Direct FCM + APNs	The app sends a raw FCM token (Android) or APNs token (iOS). The backend talks to FCM HTTP v1 and APNs HTTP/2 separately.	More control, but two integrations and more credential handling on the server. Only choose this if Expo is ruled out.

> **Design for both.** Store a `token_type` next to each token (`expo` | `fcm` | `apns`). This lets us move to option B later without a schema change. In v1 the app will only send `expo` .

END-TO-END FLOW

1. The user logs in (password, Google or Apple). The app asks for notification permission and gets an Expo push token.
2. The app calls `POST /register_device` . It calls it again on every app start and whenever the token changes. Registration must be an upsert.
3. A business event happens on the backend, for example a wallet is funded. The backend adds a notification job to a queue.
4. A queue worker loads the user's active devices, respects their preferences, and sends in batches to Expo.
5. About 15 minutes later, a worker fetches push receipts and deletes tokens that report `DeviceNotRegistered` .
6. On logout, the app calls `POST /unregister_device` . When a session is replaced (single-login rule), the backend removes that session's devices itself.

3. Push: device token endpoints

`POST` `/register_device` `P0`

Auth required. This call creates or updates the device row. Upsert on `push_token` . If the same token currently belongs to *another* user (shared phone, account switch), move it to the current user. A device must never receive pushes for an account that isn't signed in on it.

```
// Request
{
  "user_id": 123,
  "push_token": "ExponentPushToken[xxxxxxxxxxxxxxxxxxxxxx]",
  "token_type": "expo",              // "expo" | "fcm" | "apns"
  "platform": "ios",                 // "ios" | "android"
  "device_id": "6F1C-...-A9",        // stable per install, generated by the app
  "device_name": "iPhone 15 Pro",
```

```
  "os_version": "18.1",
  "app_version": "1.0.2",
  "timezone": "Africa/Lagos",
  "locale": "en-NG"
}

// Response
{ "status": true, "message": "Device registered.", "data": { "device_id": "6F1C-...-
A9" } }
```

- Link the device row to the **current auth session/token id** so it can be cleaned up when that session ends (see below).
- The app may call this many times. Duplicate calls must succeed and must not create duplicate rows.
- Validation: `push_token` required. For `expo`, it must match `^Expo(nent)?PushToken\[.+\]$`. Return 422 for a bad format.

POST **`/unregister_device`** P0

Called on logout *before* the app throws away its session token. Body: `{ "user_id": 123, "device_id": "...", "push_token": "..." }`. Delete or deactivate the row and return `status: true`, even if the row was already gone.

`Session lifecycle rules (no endpoint)` P0

- **New login replaces old session:** Mega-Sub allows one active login. When a login on device B revokes device A's token, also deactivate device A's push rows. Otherwise device A keeps getting transaction alerts for an account it's no longer signed into.
- **Password change / account deactivation / `delete_account` :** remove all of the user's device rows.
- **Optional security push:** on login from a new device, send a "New login on your account" push to the device that was just signed out, *before* deactivating it. See the `security.new_login` event.

4. Push: what triggers a notification

Each event below should send a push to the affected user. `type` is the value the app reads from `data.type` to decide which screen to open.

type	When	Example title / body	Pri.
`wallet.funded`	Naira virtual account credited / top-up settled	Wallet funded · ₦5,000 has been added to your wallet.	P0
`transaction.success`	Airtime, data, bulk, cable, electricity, result checker, eSIM purchase completed	Data purchase successful · 2GB MTN sent to 0803•••4567.	P0

type	When	Example title / body	Pri.
`transaction.failed`	Purchase failed at the provider	Airtime purchase failed · ₦500 has been refunded to your wallet.	P0
`transaction.reversed`	Pending transaction later reversed or refunded	Refund processed · ₦1,200 returned to your wallet.	P0
`transaction.pending_resolved`	A "pending" purchase becomes final (success or failure) later	Your electricity purchase is now complete. Tap to view your token.	P1
`schedule.executed`	A scheduled purchase ran	Scheduled data purchase completed.	P1
`schedule.failed`	A scheduled purchase couldn't run (low balance, provider error)	Scheduled purchase failed · Insufficient wallet balance.	P1
`schedule.upcoming_low_balance`	~24h before a scheduled purchase when the balance won't cover it	Top up to keep your scheduled purchase running.	P2
`kyc.approved` / `kyc.rejected`	BVN / card KYC review finished	Verification approved · Your limits have been upgraded.	P1
`card.funded` / `card.debited` / `card.declined`	Virtual card activity	Card payment of $12.99 at Netflix approved.	P1
`esim.ready`	eSIM provisioned and ready to install	Your eSIM is ready to install.	P1
`referral.reward`	Referral bonus credited	You earned ₦200 from a referral.	P2
`security.new_login`	Login from another device (sent to the old device)	New login on your account · If this wasn't you, contact support.	P0
`security.password_changed` / `security.pin_changed`	Password or transaction PIN changed	Your transaction PIN was changed.	P1
`promo.broadcast`	Admin marketing / announcement (only to users who opted in)	Weekend data bonanza: 10% off all MTN bundles.	P2

> **Never put sensitive data in a push.** Push text shows on the lock screen and passes through Apple, Google and Expo. Do **not** include PINs, electricity tokens, result-checker PINs, eSIM activation codes, full card numbers, full phone numbers, BVN/NIN or account balances in `title`, `body` or `data`. Send an id, and the app will fetch the details over the authenticated API.

ADMIN BROADCAST P2

An admin-panel action (not a mobile endpoint) to send a `promo.broadcast` to all users who opted into promotions, or to a segment. It must be throttled and queued: send in batches of 100 and stay under 600 notifications/second per Expo project.

5. Push: payload format

Message the backend sends to Expo, one object per device token:

```
POST https://exp.host/--/api/v2/push/send
Content-Type: application/json
Accept: application/json
Accept-Encoding: gzip, deflate
Authorization: Bearer <EXPO_ACCESS_TOKEN>     // only if "Enhanced push security" is
enabled

[
  {
    "to": "ExponentPushToken[xxxxxxxxxxxxxxxxxxxxx]",
    "title": "Wallet funded",
    "body": "₦5,000 has been added to your wallet.",
    "sound": "default",
    "priority": "high",
    "channelId": "transactions",      // Android channel, see table below
    "badge": 3,                        // iOS: user's unread count (if inbox is built)
    "ttl": 86400,
    "data": {
      "notification_id": "9b2f6c1e-...",   // UUID, used to de-duplicate
      "type": "wallet.funded",
      "user_id": 123,
      "transaction_id": 98765,           // whichever id applies: transaction_id,
      "reference": "MSB-20261017-ABC",   // schedule_id, card_id, esim_id, ...
      "created_at": "2026-10-17T09:30:00Z"
    }
  }
]
```

REQUIRED FIELDS IN `DATA`

- `notification_id`: a unique UUID per notification. The app uses it to ignore duplicates and to mark the item read in the inbox.
- `type`: one of the values in section 4. Unknown types open the Notifications screen.
- `user_id`: the app ignores a push if it doesn't match the signed-in user. This protects against stale tokens.
- The resource id needed to open the right screen: `transaction_id`, `schedule_id`, `card_id`, `esim_id`, or `kyc_type`.

ANDROID NOTIFICATION CHANNELS

The app creates these channels on startup. The backend must send the matching `channelId` so users can mute categories separately in Android settings.

channelId	Used for types	Importance
`transactions`	wallet.*, transaction.*, schedule.*, card.*, esim.*, referral.*	High
`security`	security.*, kyc.*	High
`promotions`	promo.*	Default

6. Push: sending, receipts and cleanup

- **Always send asynchronously** from a queue/job, never inside the API request that triggered the event. A slow Expo call must not delay or fail a purchase. P0
- **Batch:** up to 100 messages per request to `/push/send`. P0
- **Retry** on HTTP 429 / 5xx with exponential backoff (for example 1s, 2s, 4s… up to 5 tries). P0
- **Store the ticket** (`id`) returned for each message. After about 15 minutes, call `POST https://exp.host/--/api/v2/push/getReceipts` with up to 1,000 ids. P0
- **Clean up dead tokens:** if a ticket or receipt has `details.error = "DeviceNotRegistered"`, deactivate that token right away and stop sending to it. Expo and Apple can throttle senders that keep sending to dead tokens. P0
- Log and alert on `InvalidCredentials` (APNs/FCM credentials in EAS are wrong or expired) and `MessageTooBig` (payload over 4 KB). P1
- **Idempotency:** one business event must create one `notification_id`. Retrying a job must not send the same push twice. P1
- Keep a `notification_deliveries` table (section 12) (user, type, status, ticket id, error) so support can answer "I didn't get a notification". P1

POST `/test_push` P1 · non-production or admin-only

Sends a test notification to all of the signed-in user's devices. Body: `{ "user_id": 123, "type": "wallet.funded" }`. This lets QA check delivery and deep links on real phones without making real purchases. Disable it in production, or restrict it to staff accounts.

7. Push: preferences and in-app inbox

GET `/notification_preferences` **PUT `/notification_preferences`** P1

Needed because the Terms & Privacy screen promises users they can opt out of promotional push notifications.

```
// GET response / PUT body
{
  "status": true,
  "data": {
    "transactions": true,     // wallet, purchases, schedules, cards, eSIM
    "promotions": false,      // marketing (default OFF unless the user opted in)
    "security": true          // read-only: always true, cannot be disabled
  }
}
```

The send worker must check these before sending. `security` notifications are always sent.

GET `/notifications` P1

Right now the app's Notifications screen is built from transaction history. Wallet-funding items appear only after the user enters their PIN. A real inbox lets pushes appear there too (KYC, security, promos), and allows unread badges.

```
// GET /notifications?user_id=123&page=1&per_page=20
{
  "status": true,
  "data": {
    "items": [
      {
        "id": "9b2f6c1e-...",            // same as push data.notification_id
        "type": "wallet.funded",
        "title": "Wallet funded",
        "body": "₦5,000 has been added to your wallet.",
        "data": { "transaction_id": 98765 },
        "read_at": null,
        "created_at": "2026-10-17T09:30:00Z"
      }
    ],
    "unread_count": 3,
    "page": 1, "per_page": 20, "total": 57
  }
}
```

POST **`/notifications/read`** P1

Body: `{ "user_id": 123, "ids": ["9b2f..."] }` or `{ "user_id": 123, "all": true }`. Returns the new `unread_count`. Also add `unread_notifications` to the `/dashboard` response so the bell icon on Home can show a badge without an extra request. P2

8. Push: credentials and setup

Item	Who	Notes
Apple Push Notification key (`.p8`), Key ID, Team ID	Apple Developer account owner	Create in Apple Developer → Keys with "Apple Push Notifications service" enabled. Upload it to the EAS project (`eas credentials`). The same key works for development and production.
Firebase project + FCM V1 service-account JSON	Mobile team / Firebase owner	Create an Android app in Firebase with the final package name. Upload the service-account key to EAS (FCM V1). The app adds `google-services.json`.
Expo access token (only if "Enhanced push security" is turned on)	Expo account owner (`kalex235`) → backend	Store it as a server secret, e.g. `EXPO_ACCESS_TOKEN`. Recommended: turn it on so a leaked push token alone can't be used to send pushes.
Queue worker + scheduler	Backend	The worker sends pushes. A scheduled job (every 15 min) checks receipts.

If the backend chooses option B (direct FCM/APNs) instead, the backend needs the same `.p8` key and the FCM service-account JSON directly, plus the iOS bundle ID as the APNs topic.

9. Continue with Google (existing `POST /google`)

The app already has a Google flow (`lib/googleAuth.js`). It's currently hidden on the login screen. Native Google Sign-In gives the app an **ID token**, and the app sends it to the backend. Below is the contract the app already uses, plus the changes we need.

POST `/google` P0

No auth header. One call handles both signup and login: the backend decides whether the account is new or returning.

```
// Request (what the app sends today)
{
  "id_token": "eyJhbGciOiJSUzI1NiIs...",
  "device_name": "iOS Device",
  "referral_code": "SAMUEL12",     // optional, only for new accounts
  "tnc": true
}

// Success response (fields the app reads today, plus the ones we're asking for)
{
  "status": true,
  "message": "Login successful",
  "access": { "token": "<session token>" },
  "data": {
    "is_new_user": false,
    "requires_phone_verification": false,
    "user": {
      "id": 123,
      "email": "ada@gmail.com",
      "first_name": "Ada",
      "last_name": "Obi",
      "username": "adaobi",
      "phone_verification": true,
      "phone_number": "08031234567",   // NEW - requested
      "pin_set": true,                  // NEW - requested (see below)
      "has_password": false,            // NEW - requested
      "auth_providers": ["google"]      // NEW - requested
    }
  }
}
```

TOKEN VERIFICATION THE BACKEND MUST DO P0

1. Verify the JWT signature with Google's public keys (`https://www.googleapis.com/oauth2/v3/certs`, cached according to the response's cache headers). Or use Google's official library (e.g. `google/apiclient` `verifyIdToken`).
2. `iss` is `accounts.google.com` or `https://accounts.google.com`.

3. `aud` equals the **Web client ID**: `610379032738-gjnp8tokk3s6g160qa9l9ub3ltcdarl1.apps.googleusercontent.com`. The app passes this as `webClientId`, so tokens from *both* iOS and Android carry this audience. You may also accept the iOS and Android client IDs, but the Web ID is required.
4. `exp` is in the future (allow up to 5 minutes of clock skew).
5. `email_verified` is `true`. Otherwise return 422.
6. Identify the user by `sub` (Google's permanent user id), *not* by email. Email is only used for matching on first link (section 11).

REQUESTED CHANGES P0

- **Return `pin_set` in `data.user`.** The `/google` response doesn't say whether the user has a transaction PIN. Because of this, the app guesses from local storage on the device. That breaks when the app is reinstalled or the user moves to a new phone: a fully set-up user gets sent back to PIN setup. The `/login` and `/apple` responses need the same field.
- **Return `phone_number`** (masked is fine). The app shows it and uses it to skip phone verification.
- **Return `has_password` / `auth_providers`.** The app uses these to decide whether `delete_account` and "Change password" should ask for a current password. A Google- or Apple-only user has none. They should see "Set password" instead.
- **Session behaviour matches `/login`:** issuing a Google session must replace the previous session (single login) and trigger the same device-cleanup rules in section 3.

ERROR CODES THE APP ALREADY HANDLES

code	Meaning	Message shown to user
401	Invalid / expired / wrong-audience ID token	Backend `message`, e.g. "Invalid or expired Google token."
403	Account deactivated	"This account has been deactivated. Please contact support."
409	The email belongs to an account already linked to a *different* Google identity	"This email is already linked to a different account…"
422	Missing data (no email, email not verified, tnc false)	Backend `message`
503	Google verification temporarily unavailable	"Google sign-in is not available right now…"

Put the code in the JSON `code` field. The HTTP status may be 200 or the same code. Note: this endpoint is unauthenticated, so a 401 here won't log the user out.

10. Continue with Apple (new `POST /apple`)

> **Why this is required.** App Store Review Guideline 4.8: any iOS app that offers a third-party login such as Google must also offer an equivalent privacy-focused option. In practice that means Sign in with Apple. Without `/apple`, the iOS build with "Continue with Google" will likely be rejected.

The app will use `expo-apple-authentication` on iOS. It shows the native Apple sheet and gets an **identity token** (a JWT) and an **authorization code**. The button will be iOS-only; Android users get Google and email/password.

POST `/apple` P0

```
// Request
{
  "identity_token": "eyJraWQiOiJXNldjT0tC...",
  "authorization_code": "c1a2b3...", // single use, ~5 min
  "nonce": "f3a9...raw-nonce",        // raw; app sends sha256(nonce) to Apple
  "first_name": "Ada",                // first authorization only
  "last_name": "Obi",                 // first authorization only
  "email": "x@privaterelay.appleid.com",  // may be private relay
  "device_name": "iOS Device",
  "referral_code": "SAMUEL12",        // optional, new accounts only
  "tnc": true
}

// Response: SAME shape as /google (access.token, data.is_new_user,
// data.requires_phone_verification, data.user{..., pin_set, has_password,
// auth_providers:["apple"]}) and the same error codes.
```

TOKEN VERIFICATION THE BACKEND MUST DO P0

1. Verify the signature with Apple's public keys at `https://appleid.apple.com/auth/keys`. Match by `kid` and cache the keys.
2. `iss` = `https://appleid.apple.com`.
3. `aud` = the iOS **bundle ID** (see open question 1: currently `com.anonymous.megasub`).
4. `exp` is in the future.
5. `nonce` claim equals `sha256_hex(request.nonce)`. This stops a token from being replayed.
6. Identify the user by `sub`. Read `email` and `email_verified` from the token, not from the request body.

APPLE-SPECIFIC BEHAVIOUR P0

- **Name and email arrive only once.** Apple sends the user's name to the app only on the very first authorization. The backend must save `first_name` / `last_name` on that first call. If a new user arrives without a name (for example, the first call failed after Apple returned), create the account anyway and return `data.requires_profile_completion: true`. The app will then ask for the name.
- **Private relay emails.** Users can hide their email. You'll get `xyz@privaterelay.appleid.com`. Treat it as a real, verified email. To make transactional emails (OTPs, receipts) reach these users, register Mega-Sub's sending domain(s) and From addresses under *Apple Developer → Services → Sign in with Apple for Email Communication*, and set up SPF/DKIM.
- **Exchange the authorization code for a refresh token.** Call `POST https://appleid.apple.com/auth/token` with `grant_type=authorization_code` and a `client_secret`, which is a JWT (ES256) signed with the Sign in with Apple `.p8` key. Store the returned `refresh_token` encrypted. You need it to revoke access.
- **Revoke on account deletion** (App Store requirement 5.1.1(v)). When `delete_account` runs for a user with an Apple identity, call `POST https://appleid.apple.com/auth/revoke` with the stored refresh token.

POST `/apple/notifications` P1 · server-to-server

A public endpoint you register in Apple Developer as the "Server-to-Server Notification Endpoint". Apple POSTs a signed JWT (`payload`) here. Verify it the same way as the identity token. Handle these event types:

- `consent-revoked` / `account-delete` : unlink the Apple identity and end that user's sessions. If Apple was their only sign-in method, mark the account so the user must recover it by email.
- `email-disabled` / `email-enabled` : the user turned relay forwarding off or on. Flag the email as unreachable or reachable.

APPLE CREDENTIALS NEEDED BY THE BACKEND

Item	Where
Team ID	Apple Developer → Membership
Sign in with Apple key (`.p8`) + Key ID	Apple Developer → Keys → enable "Sign in with Apple". This is a different key from the push key and can only be downloaded once.
Client ID	The iOS bundle ID (the App ID must have the "Sign in with Apple" capability)
Registered email sources	Mega-Sub sending domains / addresses for the private relay

11. Shared rules for social accounts

- **Matching an existing account** P0 :
  1. Look up `(provider, sub)` in `social_accounts` . If found, log that user in.
  2. Otherwise, if the provider says the email is verified and a user with that email exists, **link** the new identity to that user and log them in. This covers someone who signed up with email/password and later taps "Continue with Google".
  3. Return 409 only if that user is already linked to a *different* `sub` for the same provider.
  4. Otherwise, create a new user (`is_new_user: true`).
- **New social users still finish onboarding** P0 : phone verification and transaction PIN. Set `requires_phone_verification: true` until both are done. `pin_set` tells the app which step is left. Skip email OTP verification, because the provider already verified the email.
- **Username** P1 : generate a unique username automatically for social signups (for example, from the email prefix plus digits). The app shows `username` on Home.
- **Referral codes and T&C** P0 : apply `referral_code` only when creating a new account; ignore it for existing users. Record T&C acceptance (`tnc`) with a timestamp.
- **Password login for social-only users** P1 : if someone tries `/login` with the email of an account that has no password, return 422 with a clear message, e.g. "This account uses Google sign-in. Tap Continue with Google, or reset your password to create one." Make sure `/reset` lets them set a first password.
- **`delete_account`** P0 : must accept requests without `password` when `has_password` is false (the app already sends it that way). Also revoke Apple tokens and delete push devices.
- **Rate limiting** P1 : throttle `/google` and `/apple` per IP (for example 10/min), the same as `/login` .

12. Database changes (suggested)

```
user_devices
  id, user_id (FK), session_token_id (FK, nullable), device_id, platform
ENUM('ios','android'),
  token_type ENUM('expo','fcm','apns'), push_token (UNIQUE), device_name, os_version,
  app_version, timezone, locale, is_active BOOL, last_seen_at, created_at, updated_at

notifications            -- inbox + audit
  id (UUID), user_id (FK), type, title, body, data JSON, read_at NULL, created_at

notification_deliveries  -- one row per device per notification
  id, notification_id (FK), user_device_id (FK), expo_ticket_id, status
ENUM('queued','sent','ok','error'),
  error_code NULL, sent_at, receipt_checked_at

notification_preferences
  user_id (PK/FK), transactions BOOL DEFAULT 1, promotions BOOL DEFAULT 0, updated_at

social_accounts
  id, user_id (FK), provider ENUM('google','apple'), provider_user_id (the 'sub'),
  email, is_private_email BOOL, apple_refresh_token (encrypted, NULL), created_at,
updated_at
  UNIQUE(provider, provider_user_id)

users
  + password NULLABLE (social-only accounts)    + tnc_accepted_at
```

13. Open questions for the backend / product team

#	Question	Why it matters
1	**What are the final bundle ID / package name?** The project is currently built as `com.anonymous.megasub` on both platforms. Earlier notes mention moving to `com.megasub.megasub` (iOS, to match the existing App Store record) and `com.megasub.app` (Android).	Apple's `aud` claim, the APNs topic, the FCM Android app and the Google OAuth iOS/Android clients all depend on it. If it changes later, all of them must be redone.
2	Expo Push Service (recommended) or direct FCM/APNs?	This decides what token the app sends and what credentials the backend holds.
3	Who owns the Apple Developer and Firebase accounts, and can they create the push key, the Sign in with Apple key, and the FCM project?	Neither feature can be tested on real devices without these.
4	Should promotional pushes be opt-in (default off) to match the Terms/Privacy wording and NDPR consent?	Affects the default value in `notification_preferences`.
5	Will the in-app inbox (`GET /notifications`) be built in this phase, or should the app keep building the feed from transactions?	Affects badges and how non-transaction pushes are shown.
6	Is there a staging base URL the app can point at for testing?	So QA doesn't need real money to trigger pushes.

14. Acceptance checklist

PUSH NOTIFICATIONS

- □ Registering the same token twice creates one row. Registering it under a second user moves it to that user.
- □ After logout, the device gets no more pushes for that account.
- □ Logging in on phone B stops pushes to phone A, and A receives `security.new_login`.
- □ Funding the wallet sends `wallet.funded` to iOS and Android within about 10 seconds, with the correct `channelId` and `data`.
- □ Successful, failed and reversed purchases each send the right type. No PIN, electricity token or other secret appears in any push.
- □ Uninstalling the app leads to the token being deactivated after the next receipt check.
- □ Turning off promotions stops `promo.broadcast`, while security pushes still arrive.
- □ A failing Expo call doesn't slow down or fail the purchase API.

GOOGLE / APPLE

- □ A token with the wrong `aud`, an expired token or a tampered token returns 401.
- □ An existing email/password user who taps Google is linked and logged into the *same* account (same id and wallet).
- □ A new social user gets `is_new_user: true`, `requires_phone_verification: true`, `pin_set: false`.
- □ A returning user on a freshly installed app gets `pin_set: true` and goes straight to Home.
- □ Apple sign-in with "Hide my email" works, and OTP/receipt emails reach the relay address.
- □ Apple's name is saved from the first call. A second sign-in without a name keeps it.
- □ Deleting an Apple-linked account calls Apple's revoke endpoint. A social-only account can be deleted without a password.
- □ `/login` with a social-only account's email returns the explanatory 422 message.

WHAT THE MOBILE TEAM WILL DO ONCE THESE ARE READY

Request notification permission after login, register and refresh the Expo token, create the Android channels, handle notification taps and deep links using `data.type`, call `/unregister_device` on logout, build the preferences screen, re-enable the Google button, add the Apple button on iOS (with nonce), and read `pin_set` / `has_password` from the server instead of guessing locally.